

Tp2 Consigna

Laboratorio de Programación y
Lenguajes

Trabajo Práctico N°2

Introducción a Python con prompting

Fecha de entrega: Martes 31/3 23 hs

Link: <https://classroom.github.com/a/X4xiTEDQ>

Enunciado

Dominio: Desarrollo de una mini-suite de scripts de consola para ?asistente de reseñas?
(datos de usuario + análisis simple de texto), orientada a practicar fundamentos de Python.

¿Por qué es relevante? En proyectos reales, Python se usa para automatizar tareas, procesar datos y construir lógica de negocio; además, aprender a formular prompts efectivos acelera la investigación y el aprendizaje técnico.

¿Qué van a construir?

Un repositorio con 7 ejercicios (uno por tema), cada uno implementado como un módulo Python dentro de `src/`, con funciones testeables y un set de tests unitarios en `tests/`. Además, deberán documentar un ejemplo de prompt por ejercicio, usando el patrón indicado, en un archivo `PROMPTS.md`.

Comenzar a trabajar

- Loguearse en github
- Acceder al link
- Aceptar la tarea
- Trabajar en el navegador a través de Codespaces (botón superior en el

README.md) o localmente con su IDE preferido

REQUISITOS TÉCNICOS

ESTRUCTURA:

- Tener instalado Git

Lenguajes

- [] Usar **Python 3.13+** (local puede variar, pero el CI corre con la versión configurada).
- [] No usar librerías externas fuera de las listadas en `requirements.txt`.
- [] Implementar las siguientes funciones (deben pasar los tests):
 - [] `src/variables.py`: `armar\_mensaje(nombre: str, edad: int, ciudad: str) -> str`
 - [] `src/condicionales.py`: `evaluar\_numero(n: int) -> str`
 - [] `src/listas.py`: `suma(lista: list[float]) -> float` y `promedio(lista: list[float]) -> float`
 - [] `src/funciones.py`: `contar\_palabra(texto: str, palabra: str) -> int`
 - [] `src/diccionarios.py`: `mostrar\_datos(dic: dict) -> str`
 - [] `src/loops.py`: `repetir\_palabra(palabra: str, cantidad: int) -> list[str]`
 - [] `src/operaciones.py`: `operaciones(a: float, b: float) -> dict`

Prompting:

- [] Crear el archivo `PROMPTS.md` con **7 secciones**, una por script.
- [] En cada sección incluir:
 - [] Patrón usado (según la consigna).
 - [] Prompt completo (texto).
 - [] (Opcional) Link a la conversación, si la herramienta lo permite.

Git:

- [] Realizar **? 7 commits** significativos (uno por ejercicio como mínimo).
- [] No commitear entornos virtuales (`.venv/`) ni caches.

Estructura del repositorio

La entrega debe respetar esta estructura (para que el autograding funcione):

```
.  
??? src/  
?   ??? __init__.py  
?   ??? variables.py  
?   ??? condicionales.py  
?   ??? listas.py  
?   ??? funciones.py  
?   ??? diccionarios.py  
?   ??? loops.py  
?   ??? operaciones.py
```

```
??? tests/
?   ??? __init__.py
?   ??? test_variables.py
?   ??? test_condicionales.py
?   ??? test_listas.py
?   ??? test_funciones.py
?   ??? test_diccionarios.py
?   ??? test_loops.py
?   ??? test_operaciones.py
??? .github/
?   ??? workflows/
?       ??? autograding.yml
??? requirements.txt
??? README.md
??? PROMPTS.md
```

Entrega final

Se entrega por GitHub Classroom: push a `main` del repositorio que se crea automáticamente al aceptar la tarea antes del deadline.
La corrección automática se ve en GitHub ? Actions.

DOCUMENTACIÓN

Concepto

Python

Recurso

Tipo

<https://docs.python.org/es/3.13/>

Documentación

git

<https://git-scm.com/docs>

Documentación

Machete de git

<https://education.github.com/git-cheat-sheet-education.pdf>

Guía

